

NAME: Atchaya Vharsne S

REG NO: 192524185

COURSE CODE: CSA0401

COURSE NAME: Operating systems

CO4 AT1 – Coding challenges /Hackathon

Question 1

Write a program to simulate the three file allocation methods—Contiguous, Linked, and Indexed. The program should accept inputs including the total disk size (number of blocks), the file size (number of blocks required), and the allocation method (1 for Contiguous, 2 for Linked, 3 for Indexed). Based on the selected method, the program must allocate disk blocks accordingly, display the allocated block numbers, and handle fragmentation scenarios, particularly when contiguous allocation fails due to insufficient continuous space. The output should show the list of allocated blocks or an appropriate error message if allocation is not possible. Ensure that built-in memory allocation functions are not used, and assume that disk blocks are numbered starting from 0.

```
#include <stdio.h>

#define MAX 100

int main()
{
    int disk[MAX] = {0};
    int n, fileSize, choice;
    int i, count = 0, start = -1;
    printf("Enter total disk size: ");
    scanf("%d", &n);
    printf("Enter file size: ");
    scanf("%d", &fileSize);
    printf("1. Contiguous\n");
    printf("2. Linked\n");
    printf("3. Indexed\n");
```

```

printf("Enter choice: ");
scanf("%d", &choice);
if (fileSize > n)
{
    printf("Allocation not possible.\n");
    return 0;
}
/* Contiguous Allocation */
if (choice == 1)
{
    for (i = 0; i < n; i++)
    {
        if (disk[i] == 0)
            count++;
        else
            count = 0;
        if (count == fileSize)
        {
            start = i - fileSize + 1;
            break;
        }
    }
    if (start == -1)
        printf("Contiguous allocation failed: No continuous space.\n");
    else
    {
        printf("Allocated blocks: ");
        for (i = start; i < start + fileSize; i++)
            printf("%d ", i);
    }
}

```

```

    }
}

/* Linked Allocation */
else if (choice == 2)
{
    printf("Allocated blocks: ");
    count = 0;
    for (i = 0; i < n && count < fileSize; i++)
    {
        if (disk[i] == 0)
        {
            printf("%d ", i);
            count++;
        }
    }
}

/* Indexed Allocation */
else if (choice == 3)
{
    if (fileSize + 1 > n)
    {
        printf("Allocation not possible.\n");
        return 0;
    }

    printf("Index block: 0\n");
    printf("Allocated data blocks: ");
    for (i = 1; i <= fileSize; i++)
        printf("%d ", i);
}

```

```

else

    printf("Invalid choice.\n");

return 0;
}

```

☰ Console

🖨 I/O

🤖 AI Agent

Enter total disk size: 10

Enter file size: 4

1. Contiguous

2. Linked

3. Indexed

Enter choice: 1

Allocated blocks: 0 1 2 3

Example:

Disk blocks:

0 1 1 0 1 1 0 1 0 1

There are free blocks, but there is no continuous space for a file requiring 4 blocks.

Output:

Contiguous allocation failed.

Error: Insufficient continuous free space.

Result

The program successfully simulates Contiguous, Linked, and Indexed file allocation methods, displays the allocated block numbers, and reports an error when allocation is not possible.

Question 2

Develop a program to manage free disk space using a bitmap technique. The program should accept inputs including the total number of disk blocks, the initial bitmap (where 0 represents free and 1 represents allocated), and a request for allocation specifying the number of blocks required. The program must identify and allocate free blocks efficiently, update the bitmap after allocation, and also support deallocation of specified blocks with appropriate bitmap updates. The output should display the updated bitmap after allocation and deallocation operations. Ensure that the program efficiently searches for free blocks and properly handles cases where sufficient space is not available.

```
#include <stdio.h>

#define MAX 100

int main()
{
    int bitmap[MAX], n, req, i, count = 0, block;

    printf("Enter number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter initial bitmap (0-Free, 1-Allocated):\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bitmap[i]);

    printf("Enter number of blocks to allocate: ");
    scanf("%d", &req);

    for(i = 0; i < n; i++)
        if(bitmap[i] == 0)
            count++;

    if(count < req)
        printf("Allocation failed: Insufficient free blocks.\n");
    else
    {
        count = 0;
        printf("Allocated blocks: ");
        for(i = 0; i < n && count < req; i++)
```

```

{
    if(bitmap[i] == 0)
    {
        bitmap[i] = 1;
        printf("%d ", i);
        count++;
    }
}

printf("\nBitmap after allocation: ");

for(i = 0; i < n; i++)
    printf("%d ", bitmap[i]);
}

printf("\nEnter block number to deallocate (-1 to stop): ");
scanf("%d", &block);
while(block != -1)
{
    if(block >= 0 && block < n && bitmap[block] == 1)
    {
        bitmap[block] = 0;
        printf("Block %d deallocated.\n", block);
    }
    else
        printf("Invalid or already free block.\n");
    printf("Enter block number to deallocate (-1 to stop): ");
    scanf("%d", &block);
}

printf("Bitmap after deallocation: ");
for(i = 0; i < n; i++)
    printf("%d ", bitmap[i]);

```

```
    return 0;
}
```

Console	I/O	AI Agent	44.6s
---------	-----	----------	-------

```
Enter number of disk blocks: 10
Enter initial bitmap (0-Free, 1-Allocated):
1 0 1 0 0 1 0 1 0 0
Enter number of blocks to allocate: 3
Allocated blocks: 1 3 4
Bitmap after allocation: 1 1 1 1 1 1 0 1 0 0
Enter block number to deallocate (-1 to stop): 3
Block 3 deallocated.
Enter block number to deallocate (-1 to stop): -1
Bitmap after deallocation: 1 1 1 0 1 1 0 1 0 0
```

Bitmap:

- 0 → Free disk block
- 1 → Allocated disk block

Allocation: The program searches the bitmap for 0 values and changes them to 1.

Deallocation: The selected allocated block is changed from 1 back to 0.

Advantages:

1. Simple and easy to implement.
2. Efficiently tracks free and allocated blocks.
3. Requires only one bit for each disk block.

Result: The program successfully manages free disk space using a bitmap and handles both allocation and deallocation.

Question 3

Implement a program that simulates at least three disk scheduling algorithms: FCFS, SSTF, and SCAN. The program should take as input the disk request queue, the initial head position, and the disk size. It must compute the total seek time for each algorithm, display the order in which disk requests are serviced, and compare their performance. The output should include the seek sequence and total seek time for each algorithm, along with identification of the best-performing algorithm based on minimum seek time.

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

void fcfs(int q[], int n, int head)
{
    int total = 0, i, cur = head;

    printf("\nFCFS Seek Sequence: %d ", head);

    for(i = 0; i < n; i++)
    {
        total += abs(cur - q[i]);

        cur = q[i];

        printf("%d ", q[i]);
    }

    printf("\nTotal Seek Time = %d\n", total);
}

void sstf(int q[], int n, int head)
{
    int visited[MAX] = {0}, total = 0, i, j, cur = head, min, pos;

    printf("\nSSTF Seek Sequence: %d ", head);

    for(i = 0; i < n; i++)
    {
        min = 9999;

        pos = -1;
```



```

    for(j = 0; j < n; j++)
    {
        if(!visited[j] && abs(cur - q[j]) < min)
        {
            min = abs(cur - q[j]);
            pos = j;
        }
    }
    visited[pos] = 1;
    total += min;
    cur = q[pos];
    printf("%d ", cur);
}
printf("\nTotal Seek Time = %d\n", total);
}

void scan(int q[], int n, int head, int size)
{
    int a[MAX], i, j, total = 0, cur = head, temp;
    for(i = 0; i < n; i++)
        a[i] = q[i];
    for(i = 0; i < n - 1; i++)
        for(j = i + 1; j < n; j++)
            if(a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
    printf("\nSCAN Seek Sequence: %d ", head);

```

```

for(i = 0; i < n; i++)
    if(a[i] >= head)
    {
        total += abs(cur - a[i]);
        cur = a[i];
        printf("%d ", a[i]);
    }
total += abs(cur - (size - 1));
cur = size - 1;
printf("%d ", cur);
for(i = n - 1; i >= 0; i--)
    if(a[i] < head)
    {
        total += abs(cur - a[i]);
        cur = a[i];
        printf("%d ", a[i]);
    }
printf("\nTotal Seek Time = %d\n", total);
}

int main()
{
    int q[MAX], n, head, size, i;
    printf("Enter disk size: ");
    scanf("%d", &size);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request queue: ");
    for(i = 0; i < n; i++)
        scanf("%d", &q[i]);

```

```

printf("Enter initial head position: ");

scanf("%d", &head);

printf("\n--- Disk Scheduling Results ---\n");

fcfs(q, n, head);

sstf(q, n, head);

scan(q, n, head, size);

return 0;

}

```

Console	I/O	AI Agent	42.8s
---------	-----	----------	-------

```

Enter disk size: 200
Enter number of requests: 8
Enter request queue: 98 133 87 122 14 124 65 57
Enter initial head position: 53

--- Disk Scheduling Results ---

FCFS Seek Sequence: 53 98 133 87 122 14 124 65 57
Total Seek Time = 446

SSTF Seek Sequence: 53 57 65 87 98 122 124 133 14
Total Seek Time = 199

SCAN Seek Sequence: 53 57 65 87 98 122 124 133 199 14
Total Seek Time = 331

```

- **FCFS:** Services requests in the order they arrive.
- **SSTF:** Services the request closest to the current head position.
- **SCAN:** Moves the disk head in one direction, services requests, reaches the end, and then reverses direction.
- **Seek time:** Distance travelled by the disk head.
- **Best algorithm:** The algorithm with the **minimum total seek time**.

Result: The program successfully simulates FCFS, SSTF, and SCAN, calculates their total seek times, displays their seek sequences, and identifies the algorithm with the minimum seek time.

Question 4

Write a program to implement the C-SCAN and LOOK disk scheduling algorithms. The program should accept inputs including the request queue, the initial head position, and the direction of head movement (left or right). It must simulate both algorithms, calculate the total head movement required to service all requests, and compare their efficiency. The output should display the seek sequence, total head movement, and a conclusion indicating which algorithm performs better along with a brief justification. Optionally, the program may include a graphical visualization of head movement.

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

void sort(int a[], int n)
{
    int i,j,temp;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j])
            {
                temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
}

void cscan(int q[],int n,int head,int size,char dir)
{
    int a[MAX],i,total=0,cur=head;
    for(i=0;i<n;i++) a[i]=q[i];
    sort(a,n);
    printf("\nC-SCAN Seek Sequence: %d ",head);
    if(dir=='r' || dir=='R')
```

```

{
    for(i=0;i<n;i++)
        if(a[i]>=head)
        {
            total+=abs(cur-a[i]);
            cur=a[i];
            printf("%d ",a[i]);
        }
    total+=abs(cur-(size-1));
    cur=size-1;
    printf("%d ",cur);
    total+=size-1;
    cur=0;
    printf("%d ",cur);
    for(i=0;i<n;i++)
        if(a[i]<head)
        {
            total+=abs(cur-a[i]);
            cur=a[i];
            printf("%d ",a[i]);
        }
}
else
{
    for(i=n-1;i>=0;i--)
        if(a[i]<=head)
        {
            total+=abs(cur-a[i]);
            cur=a[i];

```

```

        printf("%d ",a[i]);
    }
    total+=cur;
    cur=0;
    printf("%d ",cur);
    total+=size-1;
    cur=size-1;
    printf("%d ",cur);
    for(i=n-1;i>=0;i--)
        if(a[i]>head)
        {
            total+=abs(cur-a[i]);
            cur=a[i];
            printf("%d ",a[i]);
        }
    }
    printf("\nTotal Head Movement = %d\n",total);
}

void look(int q[],int n,int head,char dir)
{
    int a[MAX],i,total=0,cur=head;
    for(i=0;i<n;i++) a[i]=q[i];
    sort(a,n);
    printf("\nLOOK Seek Sequence: %d ",head);
    if(dir=='r' || dir=='R')
    {
        for(i=0;i<n;i++)
            if(a[i]>=head)
            {

```

```
        total+=abs(cur-a[i]);

        cur=a[i];

        printf("%d ",a[i]);

    }

    for(i=n-1;i>=0;i--)

        if(a[i]<head)

        {

            total+=abs(cur-a[i]);

            cur=a[i];

            printf("%d ",a[i]);

        }

    }

    else

    {

        for(i=n-1;i>=0;i--)

            if(a[i]<=head)

            {

                total+=abs(cur-a[i]);

                cur=a[i];

                printf("%d ",a[i]);

            }

        for(i=0;i<n;i++)

            if(a[i]>head)

            {

                total+=abs(cur-a[i]);

                cur=a[i];

                printf("%d ",a[i]);

            }

    }

}
```

```

        printf("\nTotal Head Movement = %d\n",total);
    }

int main()
{
    int q[MAX],n,head,size,i;
    char dir;
    printf("Enter disk size: ");
    scanf("%d",&size);
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request queue: ");
    for(i=0;i<n;i++) scanf("%d",&q[i]);
    printf("Enter initial head position: ");
    scanf("%d",&head);
    printf("Enter direction (L/R): ");
    scanf(" %c",&dir);
    cscan(q,n,head,size,dir);
    look(q,n,head,dir);

    printf("\nConclusion: The algorithm with minimum head movement performs better.\n");
    return 0;
}

```

Console	I/O	AI Agent	38.6s
<pre> Enter disk size: 200 Enter number of requests: 8 Enter request queue: 98 122 57 85 14 122 63 95 Enter initial head position: 53 Enter direction (L/R): R C-SCAN Seek Sequence: 53 57 63 85 95 98 122 122 199 0 14 Total Head Movement = 359 LOOK Seek Sequence: 53 57 63 85 95 98 122 122 14 Total Head Movement = 177 Conclusion: The algorithm with minimum head movement performs better. </pre>			

- **C-SCAN:** Moves in one direction and, after reaching the end, jumps to the opposite end and continues servicing requests.
- **LOOK:** Moves in one direction only until the last request in that direction, then reverses without going to the physical disk end.
- **Head movement:** Calculated as the absolute difference between the current head position and the next request.
- **Best algorithm:** The one with the **minimum total head movement**.
- **Why LOOK can be better:** It avoids unnecessary movement to the physical disk ends when there are no requests there.

Result: The program successfully implements C-SCAN and LOOK, calculates total head movement, displays the seek sequence, and compares their efficiency.

Question 5

Design a program that integrates file allocation strategies with disk scheduling techniques to optimize system performance. The program should accept inputs such as the total number of disk blocks, file size (number of blocks required), disk request queue, initial head position, allocation method (Contiguous or Indexed), and disk scheduling method (FCFS or SSTF). It must allocate disk blocks based on the chosen allocation strategy, simulate disk scheduling to access the allocated blocks, calculate the total seek time, and compare performance across different combinations. The output should display allocated blocks, seek sequence, total seek time, and identify the best combination of allocation and scheduling methods. Ensure the program handles allocation failures, avoids duplicate block allocation, and maintains efficient scheduling.

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

int absval(int x)
{
    return x < 0 ? -x : x;
}

int contiguous(int disk[],int n,int size,int a[])
{
    int i,j,count;
    for(i=0;i<=n-size;i++)
    {
        count=0;
        for(j=i;j<i+size;j++)
            if(disk[j]==0) count++;
        if(count==size)
        {
            for(j=i;j<i+size;j++)
            {
                disk[j]=1;
            }
        }
    }
}
```

```

        a[j-i]=j;
    }
    return 1;
}
}
return 0;
}

int indexed(int disk[],int n,int size,int a[])
{
    int i,count=0;
    for(i=0;i<n&&count<size+1;i++)
        if(disk[i]==0)
        {
            disk[i]=1;
            if(count>0) a[count-1]=i;
            count++;
        }
    if(count<size+1) return 0;
    return 1;
}

void fcfs(int a[],int n,int head,int *total)
{
    int i,cur=head;
    *total=0;
    printf("Seek Sequence: %d ",head);
    for(i=0;i<n;i++)
    {
        *total+=absval(cur-a[i]);
        cur=a[i];
    }
}

```

```

        printf("%d ",cur);
    }
}

void sstf(int a[],int n,int head,int *total)
{
    int used[MAX]={0},i,j,min,pos,cur=head;
    *total=0;
    printf("Seek Sequence: %d ",head);
    for(i=0;i<n;i++)
    {
        min=9999;
        pos=-1;
        for(j=0;j<n;j++)
            if(!used[j]&&absval(cur-a[j])<min)
            {
                min=absval(cur-a[j]);
                pos=j;
            }
        used[pos]=1;
        *total+=min;
        cur=a[pos];
        printf("%d ",cur);
    }
}

int main()
{
    int disk[MAX]={0},a[MAX],n,size,head,alloc,sched;
    int i,total,success;
    printf("Enter total disk blocks: ");

```

```

scanf("%d",&n);
printf("Enter file size: ");
scanf("%d",&size);
printf("Enter initial head position: ");
scanf("%d",&head);
printf("1. Contiguous\n2. Indexed\nEnter allocation method: ");
scanf("%d",&alloc);
printf("1. FCFS\n2. SSTF\nEnter scheduling method: ");
scanf("%d",&sched);
if(alloc==1)
    success=contiguous(disk,n,size,a);
else
    success=indexed(disk,n,size,a);
if(!success)
{
    printf("Allocation failed: Insufficient free blocks.\n");
    return 0;
}
printf("\nAllocated Blocks: ");
for(i=0;i<size;i++)
    printf("%d ",a[i]);
printf("\n");
if(sched==1)
    fcfs(a,size,head,&total);
else
    sstf(a,size,head,&total);
printf("\nTotal Seek Time: %d\n",total);
printf("\nBest Combination: ");
if(alloc==1)

```

```

        printf("Contiguous + ");
else
        printf("Indexed + ");
if(sched==1)
        printf("FCFS\n");
else
        printf("SSTF\n");

return 0;
}

```

Console	I/O	AI Agent	20.4s	✓
---------	-----	----------	-------	---

```

Enter total disk blocks: 100
Enter file size: 5
Enter initial head position: 50
1. Contiguous
2. Indexed
Enter allocation method: 1
1. FCFS
2. SSTF
Enter scheduling method: 1

Allocated Blocks: 0 1 2 3 4
Seek Sequence: 50 0 1 2 3 4
Total Seek Time: 54

Best Combination: Contiguous + FCFS

```

Console	I/O	AI Agent	13.2s	✓
---------	-----	----------	-------	---

```

Enter total disk blocks: 100
Enter file size: 5
Enter initial head position: 50
1. Contiguous
2. Indexed
Enter allocation method: 2
1. FCFS
2. SSTF
Enter scheduling method: 2

Allocated Blocks: 1 2 3 4 5
Seek Sequence: 50 5 4 3 2 1
Total Seek Time: 49

Best Combination: Indexed + SSTF

```

Console	I/O	AI Agent	8.8s	✓
---------	-----	----------	------	---

```

Enter total disk blocks: 5
Enter file size: 6
Enter initial head position: 2
1. Contiguous
2. Indexed
Enter allocation method: 1
1. FCFS
2. SSTF
Enter scheduling method: 1
Allocation failed: Insufficient free blocks.

```